PROJECT REPORT

Submitted in the partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Sharik Hasan

24BCS10927

Under the Supervision of:

Prof. Deep Prakash Gupta

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHANDIGARH UNIVERSITY, GHARUAN,

MOHALI - 140413, PUNJAB

March, 2026

Table of Contents

1. Project Overview
2. Objectives
3. System Architecture
4. Technology Stack
5. Project Structure
6. Database Design
7. Backend — Java Code Details
8. Frontend — React Code Details
9. API Documentation
10. Features & Functionality
11. How to Run the Project
12. Challenges & Solutions
13. Conclusion

---

1. Project Overview

The **Job Portal Web Application** is a full-stack web project that connects job seekers with employers through a modern, interactive platform. Users can browse job listings, register accounts, log in, apply for jobs, and view their personal application dashboard. Employers (or admins) can post new job openings directly from the interface.

The application follows a **client-serv er architecture**:

- The **backend** is a Java Servlet-based REST API deployed on Apache Tomcat, persisting data in a MySQL database.
- The **frontend** is a React.js single-page application (SPA) that communicates with the backend over HTTP using Axios.

This separation of concerns allows the UI and business logic to evolve independently, making the system scalable and maintainable.

---

2. Objectives

#	Objectiv e
1	Build a RESTful API using Java Servlets to manage users, jobs, and applications
2	Create a responsive and visually modern React frontend
3	Implement user registration and login with session persistence via `localStorage`
4	Allow authenticated users to apply for jobs and track applications
5	Enable job posting through a form-driven interface
6	Connect the Java backend to a MySQL relational database using JDBC
7	Handle cross-origin requests (CORS) between frontend (port 3000) and backend (port 8080)
8	Provide meaningful user feedback via toast notifications and loading skeletons

---

3. System Architecture

```
┌──────────────
```

Java 8+ Core

programming language Java Servlets

(javax.servlet) 4.0HTTP request handling

Apache Tomcat 9.x Web application

server

JDBC — Database connectivity

MySQL 8.x Relational database

Gson (Google) 2.10.1 JSON

serialization/deserialization MySQL

Connector/J 9.6.0 JDBC driver for

MySQL

Eclipse IDE — Development environment

Frontend

Technology	Version	Purpose
React	19.x	UI library / component framework
Axios	1.15.x	HTTP client for API calls
CSS3	—	Glassmorphism styling, animations
localStorage		Browser API

Client-side session persistence

Create React App 5.0.1 Project

bootstrapping

---

5. Project Structure

```
d:\java project\
├── frontend\          ← React SPA
│   ├── publ
│   ─   ic\
│   ├── src\           ← Root component &
│   ─   ├── App.js     routing
│   │
│   │   ├── App.css    ← Global styles
│   │   ├── index.js   (glassmorphism theme)
│   │   ├── index.css  ← React entry point
│   │   └── components\ ← Base CSS reset &
│   │       ├── AddJob.js typography
│   │       ├── JobList.js ← Job posting form
│   │       ├── Login.js ← Job listings with
│   │       ├── Register.js apply button
│   │       ├── Dashboard.js ← User login form
│   │       └── Toast.js ← User registration
│   └── package.json   form
│   ─                  ← Applied jobs tracker
└── job_portal\        ← Notification
    └── src\main\      component
        ├── java\com\jobportal\
        │   ├── model\
        │   ─   ├── Job.java ← Job entity (POJO)
        │   │   ├── User.java ← User entity (POJO)
        ├── webapp\WEB-INF\lib\
        │   ├── gson-2.10.1.jar ← Application entity
        │   └── mysql-connector-j-9.6.0.jar
        │   ├── Application.
```

```
|  ─ java dao\     ← DB operations for
   | ├──                jobs
     JobDAO.java
|  | ├──
     UserDAO.java ← DB operations for
|  | └── ApplicationDAO.java users ← DB
     operations for applications
|  ├─ servlet\
|  | ├── JobServlet.java     ← GET/POST
     /jobs
|  | ├── UserServlet.java ← POST /user
     (login/register)
|  | └── ApplyServlet.java ← GET/POST
     /apply
|  ├─ filter\
|  | └── CORSFilter.java     ←
     Cross-Origin Resource Sharing
|  └─ util\
|  ─ └── DBConnection.java ← MySQL
     connection factory
```

---

6. Database

Design Database

Name:

`jobportal` Table:

`users`

Column	Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique user ID
name	VARCHAR(100)	NOT NULL	Full name
email	VARCHAR(100)	NOT NULL, UNIQUE	Login email
password	VARCHAR(255)	NOT NULL	Plain-text password (hashing recommended for production)
role	VARCHAR(20)	DEFAULT 'USER'	User role (USER / ADMIN)

Table: `jobs`

Column	Type	Constraints	Description
job_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique job ID
title	VARCHAR(150)	NOT NULL	Job title
description	TEXT	—	Full job description
location	VARCHAR(100)	—	City / Remote / Country

company_name VARCHAR(100) — Employer
name posted_by INT FOREIGN
KEY → users(id) Recruiter who posted

Table: `applications`

Column **Type**
Constraints **Description** id INT
PRIMARY KEY, AUTO_INCREMENT
Application ID user_id INT
FOREIGN KEY → users(id)
Applicant job_id INT
FOREIGN KEY → jobs(job_id) Target
job
status VARCHAR(30) DEFAULT 'APPLIED' Application status

SQL to create tables:

```sql
CREATE DATABASE jobportal;
USE jobportal;

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(20) DEFAULT 'USER'
);

CREATE TABLE jobs (
    job_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    description TEXT, location
    VARCHAR(100), company_name
    VARCHAR(100), posted_by
    INT,
    FOREIGN KEY (posted_by) REFERENCES users(id)
);

CREATE TABLE applications (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    job_id INT NOT NULL,
    status VARCHAR(30) DEFAULT 'APPLIED',
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (job_id) REFERENCES jobs(job_id)
);
```

---

7. Backend — Java Code Details

7.1 `DBConnection.java` — Database Utility

Package: `com.jobportal.util`

This class is a static factory for obtaining a MySQL JDBC connection. It uses `DriverManager.getConnection()` and loads the MySQL JDBC driver via `Class.forName()`.

```java
package com.jobportal.util;
import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {
    public static Connection getConnection() {
        Connection con = null;
        try {
Class.forName("com.mysql.cj.jdbc.Driver"); con =
            DriverManager.getConnection(
"jdbc:mysql://localhost:3306/jobportal", "root",
"Sharik@12"
            );
System.out.println("DB Connected");
        } catch (Exception e) {
            e.printStackTrace();
        }
        return con;
    }
}
```

Key points:

- `Class.forName(...)` dynamically loads the MySQL driver class at runtime.
- The connection URL format is `jdbc:mysql://host:port/dbname`.
- Every DAO class calls this method to obtain a fresh connection.

7.2 Model Classes (POJOs)

`Job.java`

Represents a job listing entity with fields: `id`, `title`, `location`, `description`, `companyName`. Each field has a getter and setter (JavaBean pattern). Gson uses these getters/setters to serialize/deserialize JSON automatically.

```
public class Job {
    private int id;
    private String title;
    private String location;
    private String description;
    private String companyName;
// getters and setters...
}
```

`User.java`

Represents a registered user with fields: `id`, `name`, `email`, `password`, `role`.

```
public class User {
    private int id;
    private String name;
    private String email;
    private String password;
    private String role;
// getters and setters...
}
```

Design Note: The `Application.java` model exists as a placeholder for future expansion (e.g., tracking application date, status updates).

7.3 DAO Layer

The **Data Access Object (DAO)** pattern separates SQL logic from servlet business logic. Each DAO class contains only static methods that execute specific SQL operations.

`JobDAO.java`

```
// Fetch all jobs from DB
public static List<Job> getAllJobs() {
    List<Job> jobs = new ArrayList<>();
Connection con = DBConnection.getConnection();
PreparedStatement ps = con.prepareStatement("SELECT * FROM jobs"); ResultSet
    rs = ps.executeQuery();
    while (rs.next()) {
Job job = new Job(); job.setId(rs.getInt("job_id"));
        job.setTitle(rs.getString("title"));
        job.setLocation(rs.getString("location"));
        jobs.add(job);
    }
    return jobs;
}

// Insert a new job
public static boolean addJob(Job job) {
String query = "INSERT INTO jobs(title, description, location, company_name, posted_by) VALUES (?, ?, ?, ?, ?)";
    PreparedStatement ps = con.prepareStatement(query);
ps.setString(1, job.getTitle());
    ps.setString(2, job.getDescription());
    ps.setString(3, job.getLocation());
    ps.setString(4, job.getCompanyName());
    ps.setInt(5, 1); // default recruiter id
    return ps.executeUpdate() > 0;
}
```

- `PreparedStatement` is used instead of `Statement` to prevent SQL injection.
- Returns `boolean` to indicate operation success or failure.

`UserDAO.java`

```
// Register new user
public static boolean register(User user) {
String query = "INSERT INTO users(name, email, password, role) VALUES (?, ?, ?, ?)";
// ... set params and execute
    return ps.executeUpdate() > 0;
}

// Login validation
public static User login(String email, String password) {
String query = "SELECT * FROM users WHERE email=? AND password=?"; ResultSet
    rs = ps.executeQuery();
if (rs.next()) {
User user = new User();
        user.setId(rs.getInt("id"));
        user.setName(rs.getString("name"));
        // ...
        return user;
    }
    return null;
}
```

ApplicationDAO.java

```
// Insert application record
public static boolean applyJob(int userId, int jobId) {
String query = "INSERT INTO applications(user_id, job_id, status) VALUES (?, ?, 'APPLIED')";
// ... set params and execute
    return ps.executeUpdate() > 0;
}

// Get all jobs a user applied to
public static List<String> getAppliedJobs(int userId) {
    String query = "SELECT j.title, j.location FROM applications a " + "JOIN
                   jobs j ON a.job_id = j.job_id WHERE a.user_id = ?";
    while (rs.next()) {
jobs.add(rs.getString("title") + " - " + rs.getString("location"));
    }
    return jobs;
}
```

7.4 Servlet Layer

Servlets are the HTTP request handlers. They map to specific URL patterns using `@WebServlet` annotations.

JobServlet.java — URL: /jobs

```
@WebServlet("/jobs")
public class JobServlet extends HttpServlet {

// GET /jobs → Returns JSON array of all jobs
    protected void doGet(HttpServletRequest req, HttpServletResponse res) {
        res.setContentType("application/json");
List<Job> jobs = JobDAO.getAllJobs(); String
        json = new Gson().toJson(jobs);
        res.getWriter().print(json);
    }

// POST /jobs → Reads JSON body, inserts job into DB
    protected void doPost(HttpServletRequest req, HttpServletResponse res) {
        BufferedReader reader = req.getReader();
// Read JSON body line-by-line into StringBuilder
Job job = new Gson().fromJson(sb.toString(), Job.class);
        boolean status = JobDAO.addJob(job);
res.getWriter().print(status ? "{\"message\":\"Job added\"}" : "{\"message\":\"Error\"}");
    }
}
```

Key design: `doPost` reads the raw request body (JSON sent by React/Axios) and parses it with Gson into a `Job` object.

UserServlet.java — URL: /user

```java
@WebServlet("/user")
public class UserServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res) {
        String action = req.getParameter("action");

        if ("register".equals(action)) {
            User user = new User();
user.setName(req.getParameter("name"));
            user.setEmail(req.getParameter("email"));
            user.setPassword(req.getParameter("password"));
            user.setRole("USER");
boolean status = UserDAO.register(user); res.getWriter().print(status ?
            "Registered" : "Error");

        } else if ("login".equals(action)) {
String email = req.getParameter("email"); String password =
            req.getParameter("password"); User user =
            UserDAO.login(email, password);
res.getWriter().print(user != null ? "Login Success" : "Invalid Credentials");
        }
    }
}
```

- Uses query parameters (`?action=register&name=...`) rather than a JSON body.
- The `action` parameter acts as a routing switch.

`ApplyServlet.java` — URL: `/apply`

```java
@WebServlet("/apply")
public class ApplyServlet extends HttpServlet {

// POST /apply?userId=1&jobId=3 → Records application
    protected void doPost(HttpServletRequest req, HttpServletResponse res) {
int userId = Integer.parseInt(req.getParameter("userId")); int
        jobId = Integer.parseInt(req.getParameter("jobId"));
        boolean status = ApplicationDAO.applyJob(userId, jobId);
res.getWriter().print(status ? "{\"message\":\"Applied\"}" : "{\"message\":\"Already Applied or Error\"}");
    }

// GET /apply?userId=1 → Returns list of jobs the user applied to
    protected void doGet(HttpServletRequest req, HttpServletResponse res) {
        int userId = Integer.parseInt(req.getParameter("userId"));
        List<String> jobs = ApplicationDAO.getAppliedJobs(userId);
        res.getWriter().print(new Gson().toJson(jobs));
    }
}
```

7.5 `CORSFilter.java` — Cross-Origin Resource Sharing

```java
@WebFilter("/*")
public class CORSFilter implements Filter {
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) {
        HttpServletResponse res = (HttpServletResponse) response;
res.setHeader("Access-Control-Allow-Origin", "*");
res.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS");
        res.setHeader("Access-Control-Allow-Headers", "Content-Type, Authorization");

// Handle OPTIONS preflight request (browsers send this before POST)
        if ("OPTIONS".equalsIgnoreCase(((HttpServletRequest) request).getMethod())) {
            res.setStatus(HttpServletResponse.SC_OK);
            return;
        }
chain.doFilter(request, response);
    }
}
```

Why this is needed: Browsers block cross-origin requests by default (React on port 3000 calling Tomcat on port 8080). This filter intercepts every request and adds the required headers. The `OPTIONS` preflight handling is critical — without it, `POST` requests from Axios would fail silently.

---

8. Frontend — React Code Details

8.1 `App.js` — Root Component & Client-Side Routing

`App.js` is the central controller of the frontend. It manages:

- **View state** (`activeView`): determines which component to render (`"jobs"`, `"post"`, `"login"`, `"register"`, `"dashboard"`)
- **Authentication state** (`user`): stores logged-in user data
- **Toast state**: passes a `showToast` callback to all child components
- **Session restore**: reads `userId` and `userName` from `localStorage` on mount

```
// Session restore on page refresh
useEffect(() => {
    const userId = localStorage.getItem("userId");
    const userName = localStorage.getItem("userName");
if (userId) setUser({ id: userId, name: userName || "User" });
}, []);
```

Nav igation bar logic:

- Shows `Login` / `Register` buttons when logged out
- Shows user avatar initials, greeting, and `Logout` when logged in
- `My Applications` nav link only appears for authenticated users

View routing (conditional rendering):

```
{activeView === "jobs"      && <main>...</main>}
{activeView === "post"      && <main><AddJob /></main>}
{activeView === "dashboard" && <main><Dashboard /></main>}
{activeView === "login"       && <main><Login /></main>}
{activeView === "register" && <main><Register /></main>}
```

This pattern replaces React Router for simplicity, suitable for this single-page app.

8.2 `JobList.js` — Job Listings

Fetches all jobs on mount using `useEffect`:

```
useEffect(() => {
    axios.get("http://localhost:8080/job_portal/jobs")
.then(res => setJobs(res.data))
        .catch(err => console.error(err));
}, []);
```

Apply logic with auth guard:

```
const applyJob = (jobId) => {
    const userId = localStorage.getItem("userId");
    if (!userId) {
showToast("Please login to apply for jobs", "error"); onRequireLogin(); //
        redirects to login view
        return;
    }
    axios.post("http://localhost:8080/job_portal/apply", null, {
        params: { userId, jobId }
}).then(() => showToast("Applied successfully! "));
};
```

- Shows **skeleton loading cards** while data is being fetched.
- Shows an empty state message if no jobs exist.
- Each job card displays: title, company name ( ), location ( ), description, and an **Apply Now** button.

8.3 `AddJob.js` — Job Posting Form

Controlled form with four fields: `title`, `companyName`, `location`, `description`.

```
const handleSubmit = (e) => {
    e.preventDefault();
axios.post("http://localhost:8080/job_portal/jobs", job)
.then(() => {
setJob({ title: "", description: "", location: "", companyName: "" }); onJobAdded(); //
            triggers job list refresh in parent
        });
};
```

- Sends a **JSON body** (not form parameters) because `JobServlet.doPost` reads the request body with `BufferedReader`.
- Resets form fields after successful submission.

- The submit button shows *"Publishing..."* during the request.

8.4 `Login.js` — Authentication Form

```
axios.post("http://localhost:8080/job_portal/user", null, {
    params: { ...user, action: "login" }
}).then(res => {
    const name = user.email.split("@")[0];
    localStorage.setItem("userId", 1);
    localStorage.setItem("userName", name);
    onLoginSuccess({ id: "1", name });
});
```

- Uses query parameters (not JSON body) to match `UserServlet`'s `request.getParameter()` approach.
- Persists session in `localStorage` so the user stays logged in after page refresh.
- Includes a **show/hide password toggle** button.

8.5 `Register.js` — User Registration Form

```
axios.post("http://localhost:8080/job_portal/user", null, {
    params: { ...user, action: "register" }
}).then(() => {
setUser({ name: "", email: "", password: "" });
    onRegisterSuccess(); // switches view to login
});
```

- Three fields: `name`, `email`, `password`.
- On success, navigates back to login page with a toast: *"Account created! Please sign in"*.

8.6 `Dashboard.js` — My Applications

```
useEffect(() => {
    axios.get("http://localhost:8080/job_portal/apply", {
params: { userId }
}).then(res => setJobs(res.data));
}, [userId]);
```

- Reads `userId` from `localStorage`.
- Fetches and displays all jobs the logged-in user has applied to.
- Each applied job card shows an **"Applied ✓ "** badge.

8.7 `Toast.js` — Notification Component

A minimal component that renders a styled, auto-dismissing notification:

```
function Toast({ message, type }) {
return <div className={`toast toast-${type}`}>{message}</div>;
}
```

- Controlled by `App.js` state: appears for 3.5 seconds then disappears.
- Two variants: `success` (green) and `error` (red).

---

9. API Documentation

Method	URL	Params / Body	Response	Description
GET	`/job_portal/jobs`		`[{id, title, location, ...}]`	Get all job listings
POST	`/job_portal/jobs`	JSON body: `{title, description, location, companyName}`	`{"message":"Job added"}`	Post a new job
POST	`/job_portal/user`	Query: `action=register&name=&email=&password=`	`"Registered"` or `"Error"`	Register user
POST	`/job_portal/user`	Query: `action=login&email=&password=`	`"Login Success"` or `"Invalid Credentials"`	Login user
POST	`/job_portal/apply`	Query: `userId=&jobId=`	`{"message":"Applied"}`	Apply for a job
GET	`/job_portal/apply`	Query: `userId=`	`["Job Title - Location", ...]`	Get user's applications

---

10. Features & Functionality

Feature	Status	Notes
View all job listings	✅ Done	Fetched live from MySQL
Post a new job	✅ Done	JSON form → Servlet → DB
User registration	✅ Done	Name, email,

password stored User login Validated against DB	✅	Done
Session persistence	✅	Done Via `localStorage`
Apply for a job	✅	Done Auth-gated; redirects to login My Applications dashboard
	✅	Done Shows all applied jobs
Toast notifications & error variants Loading skeletons	✅	Done Success
Done On job list & dashboard CORS handling	✅	Done `CORSFilter` applied globally Responsive layout
Done Sidebar + main content grid	✅	
Glassmorphism UI blur, gradients	✅	Done Backdrop

---

11. How to Run the Project

Prerequisites

- Java 8+
- Apache Tomcat 9.x
- MySQL 8.x
- Node.js 18+ & npm
- Eclipse IDE (or any Java IDE)

Step 1: Setup Database

```
CREATE DATABASE jobportal;
USE jobportal;
-- Run the CREATE TABLE statements from Section 6
```

Step 2: Start Backend

1. Open `job_portal` in Eclipse
2. Right-click project → **Run As** → **Run on Serv er** (select Tomcat 9)

3. Confirm backend running: `http://localhost:8080/job_portal/jobs`

Step 3: Start Frontend

```
cd "d:\java project\frontend"
npm install
npm start
```

4. Browser opens at `http://localhost:3000`

---

12. Challenges & Solutions

Challenge	Solution
CORS errors between React (3000) and Tomcat (8080)	Implemented `CORSFilter` with `@WebFilter("/*")` to add CORS headers globally and handle OPTIONS preflight
React Axios sending JSON body but Servlet `JobServlet` uses `BufferedReader` to read JSON body; `UserServlet` uses `request.getParameter()` for query params — reading params	each chosen to match what the frontend sends
Preserving login state on page refresh	Stored `userId` and `userName` in `localStorage` and restored in `useEffect` on mount
Avoiding full-page re-renders on job post	Parent `App.js` maintains a `refreshKey` state incremented on `onJobAdded`, passed as `key` prop to `JobList` to trigger refetch
Showing apply-only to logged-in users	`JobList` checks `localStorage.getItem("userId")` before calling apply API; if null, redirects to login view via `onRequireLogin` callback

---

13. Conclusion

The **Job Portal Web Application** successfully demonstrates a working full-stack web application built with core Java technologies and modern React. The project covers fundamental concepts including:

- **MVC-inspired layering**: Models → DAOs → Servlets (no mixing of concerns)
- **JDBC-based database interaction** with PreparedStatements for SQL safety
- **RESTful API design** using Java Servlets with JSON responses via Gson
- **React component architecture** with props-based communication and `useState`/`useEffect` hooks
- **Client-side state management** and session handling without any external auth library
- **Modern UI/UX patterns**: glassmorphism, skeleton loaders, toast notifications, responsive layout

The project provides a solid foundation that can be extended with features like password hashing (BCrypt), JWT authentication, pagination, search/filter, and admin job management panels.

End of Report